# 2 차 상 세 구 현 내 용

## ERD 간단 설명

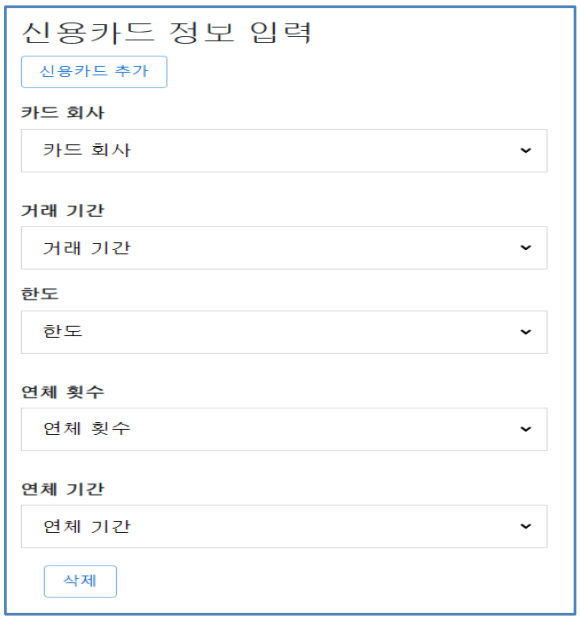
구현기능설명: **[ERD 금융정보 입력 테이블]**
금융정보 테이블은 총 4개로 CreditForm.js에서 하나의 폼으로 데이터를 입력 받습니다.
개인금융정보와 비금융 정보는 1:1 관계이고, 신용카드정보와 대출정보는 1:N 관계로 구성하였습니다.
제출 버튼 클릭 시 계산식을 통해 바로 각 컬럼에 맞는 점수 테이블에 데이터가 들어가게 됩니다.

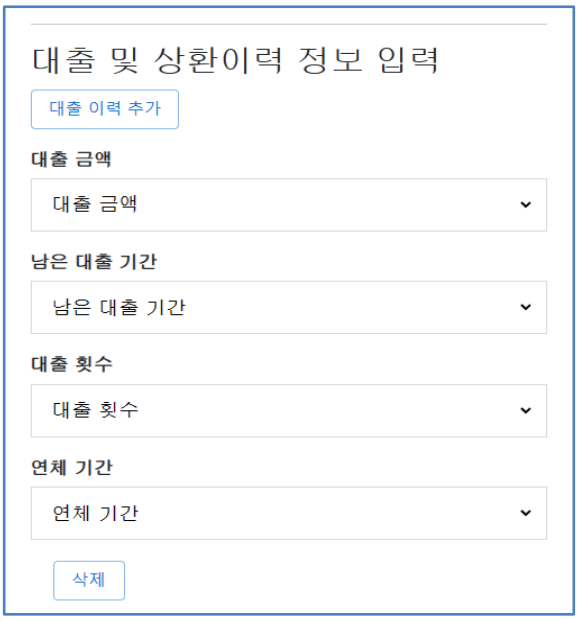
## 1:1 금융정보 입력

구현기능설명 **[1:1관계 개인금융, 비금융]**
CreditForm.js에서 정보를 입력 받고, fetch와 axios를 사용하여 HTTP POST 요청으로 서버로 전송됩니다.
서버에서는 Spring MVC를 이용하여 해당 데이터를 받아 컨트롤러를 실행합니다. 컨트롤러는 requestDto를 매개변수로 받고, Service를 호출하여 데이터를 저장하는 비즈니스 로직을 수행합니다.

또한, 비금융 정보에서는 Enum 타입을 활용하여 사용자로부터 입력 받고자 하는 데이터를 정의하였습니다. 이를 위해 JPA의 @Enumerated 어노테이션을 Entity에 적용하여 매핑하였습니다. 이를 통해 데이터의 일관성과 유효성을 유지하고, 정의된 값 범위 내에서의 입력을 보장하였습니다.

## 1:N 금융정보 입력



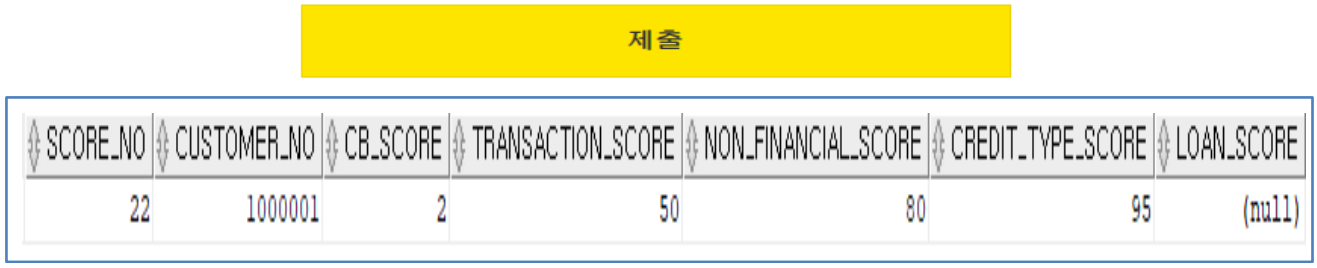




구현기능설명 **[1:N관계 신용카드, 대출정보]**
부모 컴포넌트인 CreditForm.js에서 데이터를 저장하기 위한 배열을 선언하였고, 데이터를 처리하기 위한 handleData 함수를 정의하였습니다. 또한, 추가버튼 기능만 구현하여 자식 컴포넌트가 생성되도록 하였습니다.

자식 컴포넌트인 CreditCardCompany.js와 Debt.js 에서는 데이터를 입력 받기 위한 폼을 구현하였습니다. 이때, props로 전달받은 handleData 함수를 호출하여 부모 컴포넌트로 데이터를 전달하여, 핸들링을 통해 해당 배열에 추가하거나 업데이트가 됩니다. 또한, 삭제버튼 클릭 시 부모 컴포넌트에서 해당 데이터의 배열을 제거하는 삭제기능을 구현하였습니다.

이렇게 구현된 컴포넌트들을 조합하여, 부모 컴포넌트에서 추가버튼 클릭 시 동적으로 자식 컴포넌트를 생성되고, 아코디언 형태로 펼쳐 데이터를 입력 받아 저장됩니다. 또한, SpringMVC를 이용하여 비즈니스 로직을 수행하여 입력한 데이터가 DB에 저장될 수 있도록 구현하였습니다.

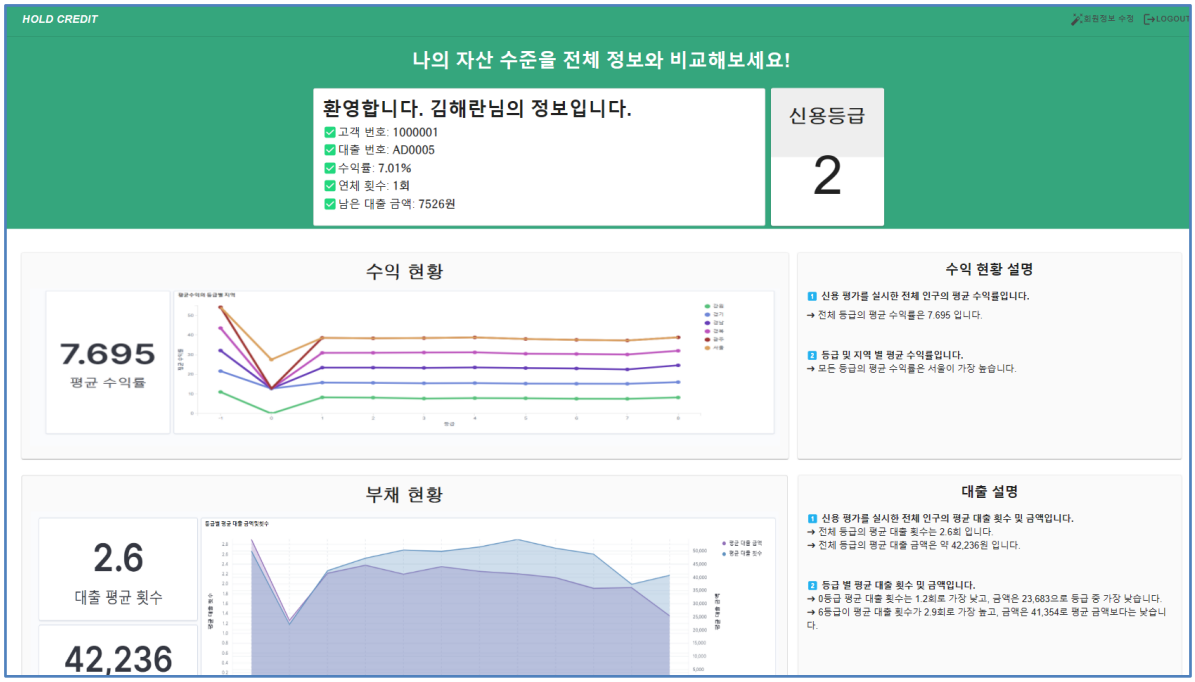
## 제출 완료시 자동 점수 산출




| SCORE_NO | CUSTOMER_NO | CB_SCORE | TRANSACTION_SCORE | NON_FINANCIAL_SCORE | CREDIT_TYPE_SCORE | LOAN_SCORE |
|---|---|---|---|---|---|---|
| 22 | 1000001 | 2 | 50 | 80 | 95 | (null) |

구현기능설명 **[자동 점수산출]**
서버로 전송된 데이터를 springMVC의 Service계층의 save메소드에서 requestDto 객체를 매개변수로 받아와 totalScore 메서드를 호출합니다. 이 메서드는 임의로 정의한 계산식을 통해 자동으로 점수를 계산하고 반환합니다.
반환된 점수는 Score엔티티에 저장이 되고, 해당 고객의 점수의 존재를 확인한 후 존재하지 않을 경우 새로운 Score엔티티를 생성하고, 존재할 경우 기존의 엔티티를 업데이트 합니다. 이렇게 구현된 코드는 전달받은 데이터를 처리하여 점수를 계산하고 엔티티에 저장하는 과정을 자동화하여 효율적인 점수 산출을 가능하게 합니다.

# ELK 빅데이터 금융정보 시각화




구현기능설명 **[ELK 빅데이터 시각화]**

임의로 생성한 금융정보 데이터를 엑셀파일로 수집하고 파이썬을 사용하여 빅데이터 분석을 수행하였습니다. 이후, 빅데이터를 머신 러닝을 활용하여 신용평가에 영향을 미칠 수 있는 주요항목을 추출하였고, 추출된 항목의 데이터를 ELK스택으로 시각화하기 위해 다음과 같은 절차를 수행하였습니다.

1. Logstash를 사용하여 엑셀 파일에서 csv 파일로 변환하여, 원하는 특정 항목만 추출하였습니다. 추출된 데이터는 구조화되어 Elasticsearch로 전송하였습니다.
2. Elasticsearch는 실시간 분산 검색 엔진으로, 적재된 데이터를 가지고 쿼리하여 복잡한 검색 및 필터링하며 필요한 집계 작업을 수행하였습니다.
3. Kibana에서는 집계된 데이터로 다양한 시각화 요소를 구성하여 대시보드를 생성하였습니다. 생성된 대시보드는 <iframe>을 통해 웹 페이지에 삽입되어 데이터 상태를 실시간으로 모니터링 할 수 있게 구현하였습니다.

.

이렇게 구현된 대시보드는 사용자 관점에서는 신용평가에 관련된 데이터를 시각적으로 확인하여 자신의 위치와 등급을 직관적으로 파악할 수 있게 도와줍니다. 또한, 운영자 관점에서는 데이터 시각화를 통해 빅데이터의 상태와 추세를 파악하고, 이를 기반으로 신용평가에 대한 결정을 내리는데 도움을 줍니다.